GeoRescue: Autonomous Multi-Agent System for Disaster Intelligence and Routing

Abstract: This document serves as a comprehensive architectural blueprint and implementation guide for the GeoRescue project. Designed for deployment on AMD Instinct MI300X infrastructure, this system leverages advanced LangChain-based multi-agent orchestration and hardware-accelerated multimodal inference to process complex Geographic Information Systems (GIS) workloads in real-time disaster scenarios.

1. System Architecture & Technology Stack

To achieve high-throughput geospatial intelligence, the architecture is decoupled into microservices, allowing individual agents to process heavy datasets independently while a central orchestrator manages the state.

Layer	Technologies & Frameworks	Functionality
Hardware / Inference	AMD MI300X, ROCm, vLLM, PyTorch	Native GPU acceleration for LLM routing (Llama-3) and Multimodal Vision (Qwen-VL) inference.
Agent Orchestration	LangChain, FastAPI	Stateful multi-agent workflows managing tools, RAG context, and REST API exposure.
GIS Spatial Engine	GeoPandas, NetworkX, OSMnx	Geospatial vector intersection, raster masking, and topological route generation.
Client Interface	Next.js, React, Mapbox GL JS	Cinematic, dark-themed interactive map UI displaying real-time agent execution and spatial outputs.

2. Data Acquisition Strategy (How to Gather Data)

A robust GIS application requires multi-layered spatial data. The Data Sourcing Agent must programmatically acquire these layers upon invocation.

- **Raster Imagery (Satellite/UAV):** Utilize the Sentinel Hub REST API to fetch pre- and post-disaster multispectral imagery.
- **Vector Base Maps (Roads & Infrastructure):** Utilize the OSMnx Python library. The agent dynamically downloads OpenStreetMap (OSM) street networks as topological graphs for the baseline routing.
- **Topography & Elevation (DEM):** Integrate USGS or SRTM Digital Elevation Models to calculate flood risk topology.

3. Implementation Methodology: Building the Agentic Pipeline

The system operates via a sequential, state-driven workflow managed by LangChain. The pipeline consists of four primary agentic phases:

Phase 1: Intent Parsing & Orchestration

The user inputs a query (e.g., "Assess structural damage and output an emergency route"). A Supervisor Agent extracts the Named Entities and orchestrates the sub-agents via FastAPI endpoints.

Phase 2: Vision-Based Multimodal Extraction

The Vision Agent receives the raw satellite raster. Running on the MI300X, Qwen-VL performs image segmentation, translating flooded pixel coordinates into real-world geographical coordinates (Lat/Lon) to output a GeoJSON polygon of the "Danger Zone".

Phase 3: Spatial Operations (The Math)

The Spatial Agent uses GeoPandas to perform a topological intersection. It overlays the Danger Zone polygon onto the OSM road network graph. Impassable roads are weighted out, and Dijkstra's algorithm finds the shortest safe path.

Phase 4: Output Generation

The Reporting Agent compiles the GeoJSON routes and streams it back to the Next.js frontend via Server-Sent Events (SSE) for a fluid UI experience.

4. Step-by-Step Setup Guide: AMD GPU & Infrastructure

Deploying on AMD hardware requires specific ROCm configurations to ensure PyTorch and LLM inference engines utilize the GPU correctly.

4.1 Provisioning & ROCm Configuration

1. Spin up an AMD Instinct MI300X instance using your AMD Developer Cloud credits.
2. SSH into the instance. Verify the GPU is recognized by the system using the rocm-smi command.

```
# Verify AMD GPU topology and driver status
rocm-smi
```

4.2 Installing ROCm-Enabled PyTorch and vLLM

Standard PyTorch targets CUDA. You must install the ROCm-specific wheel to leverage the MI300X.

```
# Create a virtual environment
python3 -m venv georescue_env
source georescue_env/bin/activate

# Install PyTorch for ROCm
pip3 install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/rocm6.0

# Install vLLM (compiled for ROCm)
pip install vllm
```

4.3 Deploying Models on vLLM

vLLM provides an API server which seamlessly integrates with LangChain.

```
# Spin up Llama-3 for the reasoning/orchestration agents
python3 -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Meta-Llama-3-8B-Instruct \
    --port 8000
```

4.4 Setting up the FastAPI & LangChain Backend

Initialize a FastAPI server to act as the middleware between the frontend UI and the LangChain agents.

```
pip install fastapi uvicorn langchain geopandas osmnx
```

4.5 Frontend Configuration (Next.js)

Initialize a Next.js application utilizing Tailwind CSS and React-Map-GL (Mapbox) to handle the visual mappings in a sleek, cinematic dark theme.

```
npx create-next-app@latest georescue-ui
npm install react-map-gl mapbox-gl
```

5. Conclusion

By combining advanced multi-agent design with the raw compute power of AMD Instinct GPUs, this architecture delivers an immediate, actionable GIS intelligence pipeline.